

Major Project Report on

**ADAPTIVE-ORDER STATE TRANSITIONS IN LINEAR
RNNs FOR EFFICIENT STATE TRACKING**

Submitted in partial fulfillment of the requirements for the degree of

BACHELOR OF TECHNOLOGY

in

ARTIFICIAL INTELLIGENCE

by

Utkarsh Shukla (Roll No.)

under the guidance of

Anand Kumar M



DEPARTMENT OF INFORMATION TECHNOLOGY
NATIONAL INSTITUTE OF TECHNOLOGY KARNATAKA
SURATHKAL, MANGALORE - 575025
SEPTEMBER, 2026

DECLARATION

I/We hereby *declare* that the Project Work Report entitled "***Adaptive-Order State Transitions in Linear RNNs for Efficient State Tracking***" , which is being submitted to the **National Institute of Technology Karnataka, Surathkal**, for the award of the Degree of Bachelor of Technology in Artificial Intelligence, is a *bonafide report of the work carried out by me/us*. The material contained in this Project Report has not been submitted to any University or Institution for the award of any degree.

Name of the Student (Registration Number) with Signature

(1) Utkarsh Shukla (Roll No.)

Department of Information Technology

Place : NITK, Surathkal-575025

Date : DD/MM/YYYY

CERTIFICATE

This is to *certify* that the Project Work Report entitled "***Adaptive-Order State Transitions in Linear RNNs for Efficient State Tracking***" submitted by

Name of the Student (Registration Number)

(1) Utkarsh Shukla (Roll No.)

as the record of the work carried out by him/her/them, is *accepted as the B.Tech. Project work report submission* in partial fulfillment of the requirement for the award of the degree of **Bachelor of Technology** in Artificial Intelligence.

Signature of Major Project Guide with date

Anand Kumar M

Write the designation of the Major Project Guide here
Department of Information Technology, NITK Surathkal

Chairman - DUGC

Signature with Date and Seal

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my project guide, **Anand Kumar M**, for his guidance, encouragement and critical feedback throughout this work. I also thank the Department of Information Technology, National Institute of Technology Karnataka, Surathkal, for providing the academic environment and resources that made this project possible.

I am grateful to the authors of DeltaProduct and of the `flash-linear-attention` library for releasing their code openly, which made it possible to reproduce and extend their work on modest hardware.

Utkarsh Shukla

ABSTRACT

Linear recurrent neural networks offer an efficient alternative to attention for long sequences, but their ability to track state depends on the structure of their state-transition matrix. DeltaProduct constructs this matrix at every time step as a product of n_h generalized Householder factors, where n_h is a single hyperparameter chosen by hand and applied identically to every token. Increasing n_h increases both the expressivity and the computation spent on every token, whether or not that token requires it. This project studies how many factors each token actually requires, and replaces the fixed global order with a per-token order that is learned.

The base method is first reproduced on the S_3 and S_5 word problems on a single 6 GB GPU, reaching a token accuracy of 0.864 at sequence length 512 after training at length 128. We then show, by explicit construction verified to machine precision, that the minimal number of factors a token requires is not its transposition length but the rank of its image under the cheapest faithful representation of the group generated by the entire input alphabet. A 5-cycle therefore requires four factors when the alphabet generates S_5 , but only two when it generates A_5 , whose three-dimensional rotation representation is unavailable to S_5 . Using a reference allocation derived from this analysis, per-token order reaches the accuracy of the maximal fixed order while requiring between 1.15 and 1.75 factors per token instead of four.

We design a monotone halting gate that scales the strength of each factor per token and train it end to end with a scheduled compute-budget penalty and no supervision on the allocation. In three of ten runs the learned gate recovers the demand structure exactly, assigning two factors to low-demand tokens and four to high-demand tokens with no under-allocation, at 2.20 required factors per token and a token accuracy of 0.9998, compared with 0.9696 for a fixed three-factor model. Training on these tasks is bistable across runs, so results are reported as the fraction of repeated runs that succeed. The reported savings are measured in required factors per token; converting them into wall-clock savings requires ragged execution of the factor product, which is identified as the principal direction for future work.

Keywords— Linear Recurrent Neural Networks, State Tracking, DeltaProduct, Householder Products, Adaptive Computation, Group Representations

CONTENTS

LIST OF FIGURES	iv
LIST OF TABLES	v
LIST OF ABBREVIATIONS	vi
1 INTRODUCTION	1
1.1 Overview	1
1.2 Motivation	2
1.3 Organization of the Report	3
2 LITERATURE SURVEY	4
2.1 Review of Research Papers	4
2.1.1 Linear Attention and the Delta Rule	4
2.1.2 Scaling Delta-Rule and Selective Recurrences	5
2.1.3 Negative Eigenvalues and Householder Products	5
2.1.4 Theory of State Tracking in Linear Recurrences	6
2.1.5 Rank and Representation	6
2.1.6 Learned Halting	7
2.1.7 Allocation with Predictable Cost	8
2.2 Outcome of Literature Survey	8
2.3 Problem Statement and Objectives	9
2.3.1 Problem Statement	9
2.3.2 Objectives	9
3 METHODOLOGY	11
3.1 System Overview	11
3.2 Task Formulation: Group Word Problems	12
3.3 Base Architecture	13
3.4 Minimal Order Requirement of a Token	13
3.5 The Adaptive-Order Layer	15
3.6 Reference Allocation	16

4	EXPERIMENTAL RESULTS AND DISCUSSION	17
4.1	Experimental Setup and Metrics	17
4.2	Reproduction of the Base Method	18
4.3	Verification of the Order-Requirement Analysis	19
4.4	Capability at Matched Compute	20
4.5	Alphabet Closure and Training Dynamics	22
4.6	End-to-End Training of the Halting Gate	24
4.7	Discussion	26
5	CONCLUSION AND FUTURE WORK	27
5.1	Conclusion	27
5.2	Future Work	28
	APPENDIX I	29
A.1	Theoretical Results	29
A.2	Matched-Compute Sequence Accuracy	29
A.3	Alphabet-Closure Details	30
A.4	Learned-Gate Details	32
A.5	Implementation Details	33
	KEY REFERENCES	36
	TIMELINE OF THE PROJECT	38

LIST OF FIGURES

3.1.1 Overview of the study	11
3.2.1 The group word problem on five items	12
3.5.1 The adaptive-order layer	16
4.2.1 Length generalisation of the reproduced DeltaProduct model on S_5 .	19
4.4.1 Token accuracy against required order per token	21
4.4.2 Required order and measured training time at $p = 0.10$	22
4.5.1 Escape fraction over ten runs per configuration	23
4.6.1 Mean factors allocated per token by the learned gate	24
A.3.1Token accuracy by alphabet and order, one run per cell	30
A.3.2Outcomes of ten runs of S5_c5_t1 at four factors	31
A.4.1Factors allocated to each requirement class by the learned gate	32
A.4.2Learned-gate runs across training configurations	33

LIST OF TABLES

2.2.1 Comparison of related methods	8
3.2.1 Alphabets used in the experiments	13
3.4.1 Order requirement of the element classes of A_5	15
4.1.1 Training configuration	17
4.2.1 Reproduction of DeltaProduct on the S_3 and S_5 word problems (the S_5 result at length 128 is measured on the first 128 positions of the length-512 evaluation)	18
4.3.1 Verification of the analysis by explicit construction (float64)	20
4.4.1 Token accuracy at matched compute (one run per cell)	21
4.5.1 Outcomes over ten runs per configuration (escape: token accuracy ≥ 0.90)	23
4.6.1 Learned allocation at $p = 0.10$, all runs ($\bar{n}$: mean factors per token with hard gates)	24
4.6.2 All learned-gate runs by training configuration	25
4.7.1 Summary of the evidence for each hypothesis	26
A.1.1 Order requirement across groups	29
A.1.2 Exact reachability of P_g with K factors, by transposition length ℓ ($\checkmark$ reachable)	29
A.2.1 Sequence accuracy at matched compute (one run per cell)	30
A.3.1 Token accuracy by alphabet and order (one run per cell; sequence accuracy in parentheses)	31
A.3.2 S5_c5_t1 : token accuracy under extended training (one run per cell)	31
A.5.1 Effect of the precision of the halting nonlinearity	34
A.5.2 Hardware and software environment	35
T.1 Project timeline	38

LIST OF ABBREVIATIONS

ACT	Adaptive Computation Time
bf16	bfloat16, 16-bit brain floating-point format
CALM	Confident Adaptive Language Modeling
CUDA	Compute Unified Device Architecture
fp32	32-bit single-precision floating-point format
GELU	Gaussian Error Linear Unit
GPU	Graphics Processing Unit
LR	Learning Rate
MLP	Multi-Layer Perceptron
MoD	Mixture-of-Depths
RNN	Recurrent Neural Network
SSM	State-Space Model
VRAM	Video Random-Access Memory
WSL	Windows Subsystem for Linux

Notation

S_n, A_n	Symmetric group on n items; alternating (even) subgroup
Σ	Input alphabet: the set of group elements a token may take
$\langle \Sigma \rangle$	Group generated by the alphabet (its closure under composition)
$\ell(g)$	Transposition length: fewest transpositions whose product is g
ρ	A representation: a homomorphism from a group to invertible matrices
$D(g)$	Order requirement of element g (Definition 3.1)
n_h	Number of Householder factors per token (the order)
β	Strength of a generalized Householder factor, $\beta \in [0, 2]$
$\lambda_{t,i}$	Continue probability emitted by the halting gate for factor i at token t
$g_{t,i}$	Monotone gate applied to factor i at token t
n_t	Effective order of token t : the number of factors it uses
γ	Weight of the compute-budget penalty
p	Mixture rate: probability that a drawn token is a high-demand 5-cycle
$\mathbb{E}[D]$	Mean order requirement over the data: the cost of an exact allocation

CHAPTER 1

INTRODUCTION

1.1 Overview

Sequence models must frequently maintain a running summary of everything they have read: the current value of a variable in a program, the position of an object after a series of moves, or the state of a finite automaton after a string of inputs. This capability is known as *state tracking*. Transformers address sequence modelling by attending over the full history at every step, which gives them great flexibility but makes their cost grow quadratically with sequence length. Linear recurrent neural networks, including linear attention [1] and selective state-space models [2], instead compress the history into a fixed-size state that is updated once per token. Their cost is linear in the sequence length and their memory during generation is constant, which makes them attractive for long-context and streaming applications.

Whether such a model can track state depends almost entirely on its *state-transition matrix*, the matrix that maps the state at step $t - 1$ to the state at step t . Early linear RNNs used diagonal transitions with non-negative entries, and a sequence of theoretical results has shown that this choice places hard limits on what they can track [3, 4, 5]. Two developments relaxed those limits. The delta rule [6, 7, 8] replaces the diagonal transition with a generalized Householder matrix, a rank-one correction of the identity; allowing that correction to reach a reflection, which introduces negative eigenvalues, lets the model represent operations such as parity [9]. DeltaProduct [10] then builds each transition as the product of n_h generalized Householder factors, obtained by applying n_h delta-rule updates per token, and shows that a larger n_h lets the model track progressively harder groups, up to the symmetric group S_5 .

In DeltaProduct the order n_h is a single hyperparameter chosen by hand and applied identically to every token. Each additional factor adds computation for every token in the sequence. Yet different inputs plainly require different amounts of expressivity: a token that swaps two items is a simpler operation than one that cycles all five. A fixed global order must therefore be set for the hardest token the model will ever see, and that cost is paid on every token regardless.

This project replaces the fixed global order with a per-token order that the model learns. A small, monotone halting gate reads each token’s representation and decides how many of the available factors to apply to it, and a compute-budget penalty encourages the gate to use as few as the task allows. Building this mechanism required answering a more basic question first: *how many factors does a token actually require?* The answer depends not on the token alone but on the whole set of inputs the model is trained on, and that analysis forms a central part of this report.

1.2 Motivation

The practical motivation is efficiency. Every Householder factor in DeltaProduct costs an additional delta-rule update per token. When most tokens in a sequence require few factors and only some require many, a fixed order spends most of its computation on tokens that do not need it. An allocation that follows each token’s requirement would pay the average requirement rather than the worst case.

The second motivation is that the correct value of n_h is difficult to choose in advance. Chapter 3 shows that the number of factors a token requires is determined by the algebraic structure of the entire input alphabet, and that adding a single, individually cheap input symbol can double the requirement of other symbols. A practitioner therefore cannot read the right order off the inputs one at a time. Choosing too small an order causes the model to fail on the tokens that exceed it; choosing too large an order wastes computation everywhere. A layer that learns its own order per token removes this hyperparameter from the design.

The third motivation is scientific. Adaptive computation has been studied extensively for the depth of Transformer and recurrent networks [11, 12, 13, 14], but not for the internal structure of the state transition of a linear RNN. Working at this level exposes a precise, verifiable relationship between the operation a token performs and the computation it needs.

1.3 Organization of the Report

Chapter 2 surveys the literature on linear recurrent architectures, the theory of state tracking and adaptive computation, and states the problem and objectives. Chapter 3 formulates the task, derives the minimal order a token requires and describes the adaptive-order layer. Chapter 4 presents the experimental setup and results, from the reproduction of the base method to the end-to-end training of the halting gate. Chapter 5 concludes and outlines future work. Appendix I collects supplementary figures and tables, and the report closes with the key references and the project timeline.

CHAPTER 2

LITERATURE SURVEY

2.1 Review of Research Papers

The papers reviewed below fall into four groups: linear recurrences built on the delta rule, the architectures that extend it to track groups, the theory of what such recurrences can track, and methods for allocating computation adaptively. Each group is discussed paper by paper and closes with a short discussion of what it implies for this project.

2.1.1 Linear Attention and the Delta Rule

Katharopoulos et al. [1] observed that if the softmax in attention is replaced by a kernel feature map, attention can be computed as a running sum over a matrix-valued state, $S_t = S_{t-1} + v_t k_t^\top$. The resulting model, the linear Transformer, is equivalent to a recurrent network during autoregressive generation, with constant cost and memory per token. Its weakness is that the state can only accumulate: nothing is ever removed, so old associations interfere with new ones as the sequence grows.

Schlag et al. [6] reinterpreted this state as a set of *fast weights* that the network writes and reads at every step, and identified the purely additive update as the cause of the capacity problem. They proposed the *delta rule*, which first removes the value currently stored under a key and then writes the new one:

$$S_t = S_{t-1} (I - \beta_t k_t k_t^\top) + \beta_t v_t k_t^\top, \quad \|k_t\| = 1. \quad (2.1)$$

The transition $I - \beta_t k_t k_t^\top$ is a *generalized Householder matrix* [15]: it leaves every direction orthogonal to k_t unchanged and scales the component along k_t by $1 - \beta_t$.

Discussion. These two papers establish the matrix-valued recurrent state that every model in this project uses, and the delta rule’s Householder transition, which is the building block whose count per token this project makes adaptive.

2.1.2 Scaling Delta-Rule and Selective Recurrences

Yang et al. [7] made the delta rule practical at scale. The recurrence in Equation 2.1 is sequential, so the authors derived a chunk-wise parallel form based on a compact representation of products of Householder matrices, allowing the model, DeltaNet, to be trained efficiently on GPUs and to compete with Transformers as a language model.

Gu and Dao [2] proposed Mamba, a state-space model whose parameters depend on the input, so that the model can selectively retain or discard information. Its transitions are diagonal, which keeps them cheap and allows a hardware-efficient parallel scan, and it established linear-time recurrences as serious language models.

Yang, Kautz and Hatamizadeh [8] combined the two ideas in Gated DeltaNet. A data-dependent decay gate, as in Mamba, erases memory quickly, while the delta rule performs targeted updates; the authors show the mechanisms are complementary.

Discussion. This line of work shows that the structure of the transition is the central design choice: diagonal transitions are fast but limited, and Householder transitions are more expressive while remaining efficient to train. The implementation used in this project builds on the chunk-wise kernels introduced here.

2.1.3 Negative Eigenvalues and Householder Products

Grazzi et al. [9] showed that restricting the eigenvalues of the transition to $[0, 1]$, as most linear RNNs do, prevents them from solving even parity. Allowing the strength β in Equation 2.1 to range over $[0, 2]$ gives the Householder factor an eigenvalue in $[-1, 1]$; at $\beta = 2$ the factor is a reflection. With this change, linear RNNs solve parity, and products of generalized Householder matrices can implement general finite automata.

Siems et al. [10] built on this result in DeltaProduct. Each token applies n_h delta-rule updates, each with its own key, value and strength, so the state-transition matrix becomes

$$A_t = \prod_{i=1}^{n_h} (I - \beta_{t,i} k_{t,i} k_{t,i}^\top), \quad (2.2)$$

which interpolates between DeltaNet ($n_h = 1$) and a dense transition. The authors

evaluate the word problems of S_3 , S_4 , A_5 and S_5 , show that increasing n_h lets a single layer track progressively harder groups, and report that the trained models extrapolate to sequences longer than those seen during training. The order n_h is fixed for the whole model and applied to every token; the paper’s discussion of future work names per-token adaptive selection of the number of factors, in the spirit of Adaptive Computation Time, as an open direction.

Discussion. DeltaProduct is the base method of this project. Its central parameter, n_h , sets both the expressivity and the cost of every token, and the paper itself identifies adapting it per token as open.

2.1.4 Theory of State Tracking in Linear Recurrences

Merrill, Petty and Sabharwal [3] showed that commonly used state-space models, like Transformers, belong to a restricted circuit complexity class. Under standard complexity-theoretic assumptions they therefore cannot solve the word problem of S_5 , which is complete for a larger class. The word problem of a finite group has since become the standard benchmark for state tracking.

Sarrof, Veitsman and Hahn [4] characterised the formal languages that state-space models can recognise and identified non-negative transition values as a key source of their limitations, consistent with the parity result above.

Shakerinava et al. [5] characterised exactly which groups diagonal state-space models can track, relating the number of layers to the length of a solvable series of the group. They also report that multi-layer models often fail to *learn* state-tracking solutions they can *represent*, a gap between expressivity and learnability.

Discussion. These results explain why Householder transitions matter and why group word problems are the right test bed. The gap between what a model can represent and what it learns is also observed in this project.

2.1.5 Rank and Representation

Larson [16] trains matrix-valued memories on group composition under a single-state bottleneck and reports that the rank the memory recruits equals the dimension of the group’s smallest faithful real representation, across five groups on both sides

of the solvable/non-solvable divide.

Howe [17] studies Householder-product linear RNNs and states that the minimal number of factors per token equals the largest reflection length among the task’s generators in a representation fixed by the task format. The paper notes that this requirement is relative to the representation, and separately reports that removing the additive input pathway of the recurrence improves length generalisation.

Discussion. Both papers relate the resources a model needs to the representation theory of the task. The analysis in Chapter 3 is related but differs in two respects that matter for adaptive computation: the representation is determined by the group generated by the input alphabet rather than by the output format, and the requirement is defined per token as a minimum over faithful representations, so that it can vary between tokens and be allocated individually.

2.1.6 Learned Halting

Graves [11] introduced Adaptive Computation Time (ACT), which lets a recurrent network perform a variable number of internal steps per input. A sigmoid halting unit emits a halting probability at each step, computation stops once the accumulated probability approaches one, and a ponder cost proportional to the number of steps is added to the loss. Graves notes that behaviour is sensitive to the weight of this cost.

Dehghani et al. [12] applied ACT per position in the Universal Transformer, a Transformer whose layers share weights and are applied recurrently, so that each position decides how many times the shared block is applied to it.

Banino, Balaguer and Blundell [13] reformulated halting probabilistically in PonderNet. Every step receives gradient, avoiding the biased gradient of ACT, and the halting distribution is regularised towards a geometric prior that keeps all step counts in play.

Discussion. These papers supply the monotone halting mechanism and the compute penalty that the adaptive-order layer adapts, and they document its sensitivity to the penalty weight, which is also studied in this project.

2.1.7 Allocation with Predictable Cost

Schuster et al. [18] proposed Confident Adaptive Language Modeling (CALM), which exits early from the decoder once a confidence measure exceeds a threshold calibrated to guarantee output quality.

Raposo et al. [14] proposed Mixture-of-Depths, which routes only the top- k tokens of each block through the computation and lets the rest skip it. The fixed capacity k is chosen explicitly so that tensor shapes stay static and the computation graph is known in advance, which makes the savings realisable on GPUs.

Discussion. Later work has favoured allocation mechanisms whose cost is known in advance, because a variable amount of computation per token is otherwise difficult to execute efficiently. The same consideration governs how the savings of this project can be realised (Chapter 5).

2.2 Outcome of Literature Survey

Table 2.2.1 summarises the methods reviewed along the dimensions relevant to this project.

Table 2.2.1: Comparison of related methods

Method	Transition per token	Order per token	Allocation
Linear attention [1]	Identity (additive only)	–	–
Mamba [2]	Diagonal, input-dependent	–	–
DeltaNet [7]	One Householder factor	Fixed, 1	–
Gated DeltaNet [8]	Gated Householder factor	Fixed, 1	–
Negative eigenvalues [9]	Householder, $\beta \in [0, 2]$	Fixed	–
DeltaProduct [10]	n_h Householder factors	Fixed, global	Hand-chosen
ACT / PonderNet [11, 13]	– (network depth)	Variable	Learned halting
Mixture-of-Depths [14]	– (block skipping)	Variable	Top- k router
This work	$\leq n_h$ Householder factors	Variable	Learned halting

Three observations follow. First, the expressivity of a linear RNN is governed by the structure of its state transition, and products of generalized Householder factors with strengths in $[0, 2]$ are currently the most expressive structure that remains efficient to train [9, 10]. Second, in every such architecture the number of factors is a global constant, so the computation spent on a token does not depend on what the token does. Third, adaptive computation is well established for network depth, where learned halting and routing are both mature, but it has not been applied to the order of the state transition. The theoretical work also indicates that the number of factors required depends on algebraic properties of the task [10, 17, 16], which suggests that the requirement varies across tokens in a way that can be characterised. Together these observations identify the gap addressed here: a DeltaProduct-family layer in which the number of factors is chosen per token by a learned mechanism, together with an account of how many factors each token requires.

2.3 Problem Statement and Objectives

2.3.1 Problem Statement

DeltaProduct applies a fixed number of Householder factors to every token, although tokens differ in the expressivity they require. The number each token requires has not been characterised, and no mechanism exists for allocating it per token. This project characterises the minimal number of factors a token requires and develops a learned, per-token halting mechanism that allocates factors accordingly, evaluated on group state-tracking tasks.

2.3.2 Objectives

- (1) To reproduce DeltaProduct on group word problems in the available computing environment, as the baseline for every later result.
- (2) To characterise theoretically the minimal number of generalized Householder factors a token requires, and to verify the characterisation by explicit construction.

- (3) To measure the capability of per-token order allocation at matched compute, using a reference allocation derived from the characterisation.
- (4) To design and implement a monotone, per-token halting gate for the number of factors, trained end to end with a compute-budget penalty and without supervision on the allocation.
- (5) To evaluate the trainability and reliability of the base layer and of the learned gate across repeated runs.

CHAPTER 3

METHODOLOGY

3.1 System Overview

The study proceeds in stages, shown in Figure 3.1.1. A data generator produces group word problems whose input alphabet, and therefore whose per-token requirement, is controlled precisely. A single DeltaProduct layer processes each sequence and a linear readout predicts, after every token, the group element obtained so far. The layer either applies a fixed number of Householder factors to every token, receives its allocation from a reference derived from the theoretical analysis, or learns its allocation through a halting gate trained with a compute-budget penalty.

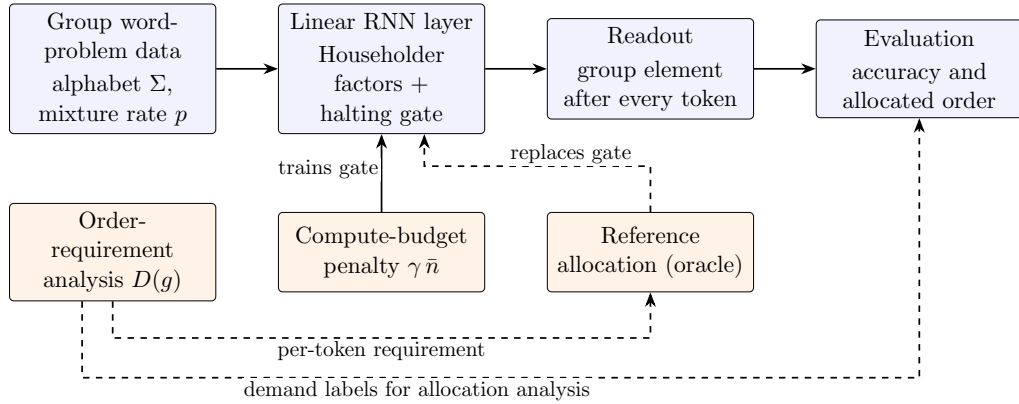


Figure 3.1.1: Overview of the study. Solid arrows are the model and its training signal; dashed arrows carry information from the theoretical analysis, which is used only for the reference allocation and for evaluation, never as a training target for the learned gate.

The study is guided by three hypotheses:

- **H1:** On inputs whose tokens differ in the expressivity they require, an allocation that follows each token's requirement reaches the accuracy of the maximal fixed order at a lower required order per token.
- **H2:** The minimal number of Householder factors a token requires is determined by the group generated by the entire input alphabet, not by the token alone.

- **H3:** A monotone halting gate trained end to end with a compute-budget penalty can recover the per-token requirement without supervision on the allocation.

3.2 Task Formulation: Group Word Problems

Five items occupy five positions (Figure 3.2.1). The model reads a sequence of instructions, one per token, each of which rearranges the items, and after every instruction it must output the arrangement reached so far. Formally, each token x_t is a permutation in a finite group G , and the target at position t is the running product

$$y_t = x_t x_{t-1} \cdots x_1. \quad (3.1)$$

Because y_t depends on every earlier token, the task cannot be solved by inspecting any single token, and because the answer is known exactly, accuracy is unambiguous.

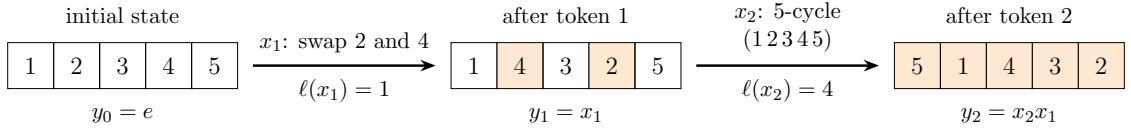


Figure 3.2.1: The group word problem on five items. Each token is a permutation; after every token the model must output the arrangement reached so far. Shaded cells are those moved by the most recent token. The transposition length ℓ counts the fewest swaps that build each token.

The *symmetric group* S_n contains all $n!$ rearrangements of n items. A *transposition* swaps two items and a *5-cycle* moves all five around one loop. The fewest transpositions that compose to g is its *transposition length* $\ell(g) = n - c(g)$, with $c(g)$ the number of cycles, so a transposition has $\ell = 1$ and a 5-cycle $\ell = 4$. The *even* permutations (even ℓ) form the *alternating group* A_n , half of S_n ; a 5-cycle is even and a transposition odd.

An *alphabet* Σ is the set of permutations the generator may emit, and the group the model must track is its *closure* $\langle \Sigma \rangle$, the set of all products of alphabet elements. An alphabet of 5-cycles contains only even permutations, so its closure is A_5 ; adding a single transposition makes odd arrangements reachable and the closure becomes S_5 . Table 3.2.1 lists the alphabets used. In the *mixture* alphabets each token is a

5-cycle with probability p and a transposition otherwise; the *closure* alphabets are sampled uniformly and are paired so that they share the same 5-cycles but differ in their closure. Sequences have 128 tokens; the output vocabulary has 127 classes (the 120 elements of S_5 and 7 special tokens).

Table 3.2.1: Alphabets used in the experiments

Alphabet	Contents	Symbols	$\langle \Sigma \rangle$	Share at $D_{\max}$
Mixture, rate p	10 transpositions, 24 five-cycles; five-cycle with probability p	34	S_5 ($p < 1$)	p
A5_c5	24 five-cycles	24	A_5	1.00
A5_c5_dt	five-cycles, double transpositions	39	A_5	1.00
A5_c5_3c	five-cycles, 3-cycles	44	A_5	1.00
S5_c5_t1	five-cycles, <i>one</i> transposition	25	S_5	0.96
S5_c5_t	five-cycles, all 10 transpositions	34	S_5	0.71

3.3 Base Architecture

The model is a single DeltaProduct layer [10] between a token embedding and a linear readout. For each token the layer computes keys $k_{t,i}$, values $v_{t,i}$ and strengths

$$\beta_{t,i} = 2 \sigma(w_i^\top h_t) \in (0, 2), \quad i = 1, \dots, n_h, \quad (3.2)$$

where h_t is the token representation, and applies n_h consecutive delta-rule steps (Equation 2.1), giving the transition of Equation 2.2; the range $(0, 2)$ lets each factor act as anything from the identity to a reflection [9]. The implementation uses the gated DeltaProduct layer of `flash-linear-attention` [19], and the readout is trained with cross-entropy over all positions.

3.4 Minimal Order Requirement of a Token

Proposition 3.1 (Rank bound). *For any strengths β_i and unit vectors k_i , $A = \prod_{i=1}^K (I - \beta_i k_i k_i^\top)$ satisfies $\text{rank}(A - I) \leq K$.*

Proof. $H_1 \cdots H_m - I = (H_1 \cdots H_{m-1} - I)H_m + (H_m - I)$, and $H_m - I$ has rank at most one, so each factor raises $\text{rank}(\cdot - I)$ by at most one. $\square$

For a permutation matrix, $\text{rank}(P_g - I) = n - c(g) = \ell(g)$, so a layer that realises token g as its permutation matrix needs at least $\ell(g)$ factors, whatever the strengths.

Proposition 3.2 (Parity bound). *If every strength is fixed at $\beta_i = 2$, then $\det A = (-1)^K$ and P_g is reachable only if $K \geq \ell(g)$ and $K \equiv \ell(g) \pmod{2}$. With strengths free in $[0, 2]$, every $K \geq \ell(g)$ suffices, since surplus factors can be set to $\beta = 0$, the identity.*

Proposition 3.2 fixes a design decision: to use fewer than its maximum number of factors, the layer must switch a factor off exactly, which requires $\beta = 0$. The halting gate therefore *scales* factor strengths rather than masking factors of fixed strength.

The role of the representation. A representation ρ of a group maps each element to an invertible matrix with $\rho(gh) = \rho(g)\rho(h)$; it is *faithful* if distinct elements map to distinct matrices. Permutation matrices are one representation, but a group generally has others of different dimensions. A layer need only keep group elements distinguishable, so it may use any faithful representation, and Proposition 3.1 then applies to $\rho(g)$; for orthogonal representations the bound is attained, since by the Cartan–Dieudonné theorem an orthogonal map is a product of $\text{rank}(\rho(g) - I)$ reflections [20].

Definition 3.1 (Order requirement). For an alphabet Σ and a token $g \in \Sigma$,

$$D(g) = \min_{\rho \text{ faithful rep. of } \langle \Sigma \rangle} \text{rank}(\rho(g) - I). \quad (3.3)$$

The minimum ranges over representations of the closure, not of the token. For alphabets generating S_5 , the four-dimensional standard representation gives $D(g) = \ell(g)$ and no faithful representation does better (Section 4.3). A_5 , however, is isomorphic to the rotation group of the icosahedron, a faithful three-dimensional representation that S_5 lacks. Every non-identity rotation fixes exactly its axis, so $\text{rank}(R - I) = 2$ for every non-identity element of A_5 (Table 3.4.1).

Hence the requirement of a token is a property of the whole alphabet: adding one transposition to an alphabet of 5-cycles changes the closure from A_5 to S_5 and raises

Table 3.4.1: Order requirement of the element classes of A_5

Element class	Count	ℓ (perm. rep.)	Rank in 3-D rep.
Identity	1	0	0
Double transposition	15	2	2
3-cycle	20	2	2
5-cycle	24	4	2

the requirement of every 5-cycle from two to four, which no model of per-token cost can produce (tested in Section 4.5). Per-token allocation can also only help where D varies; for closures A_4 and A_5 it does not (Table A.1.1, Appendix I).

3.5 The Adaptive-Order Layer

The layer keeps a maximum order K and adds a halting gate, a two-layer perceptron of hidden width 64 applied to the token representation, $z_t = W_2 \text{GELU}(W_1 h_t + b_1) + b_2 \in \mathbb{R}^K$. Its outputs become gates through a cumulative product:

$$\lambda_{t,i} = \sigma(z_{t,i}), \quad g_{t,i} = \prod_{j=1}^i \lambda_{t,j}, \quad n_t = \sum_{i=1}^K g_{t,i}, \quad \tilde{\beta}_{t,i} = g_{t,i} \beta_{t,i}. \quad (3.4)$$

The gates are monotone, $g_{t,1} \geq \dots \geq g_{t,K}$, so once the gate closes it stays closed, and n_t is the *effective order* of token t . Following Proposition 3.2, the gate scales each strength; a factor with $g_{t,i} = 0$ erases and writes nothing and is exactly the identity. The layer is shown in Figure 3.5.1; its forward pass is given as Algorithm 1 in Appendix I. During training the gates are real-valued; at evaluation they are rounded, $\hat{g}_{t,i} = \mathbf{1}[g_{t,i} > 0.5]$, so that the reported order is an integer number of factors actually applied.

Training objective. Nothing in the task loss rewards using fewer factors, so a penalty on the effective order is added,

$$\mathcal{L} = \mathcal{L}_{\text{task}} + \gamma(s) \cdot \frac{1}{BT} \sum_{b,t} n_{b,t}, \quad \gamma(s) = \gamma \cdot \min\left(1, \max\left(0, \frac{s - s_w}{s_r}\right)\right), \quad (3.5)$$

where s is the training step. The penalty is off for the first $s_w = 4000$ steps, so that

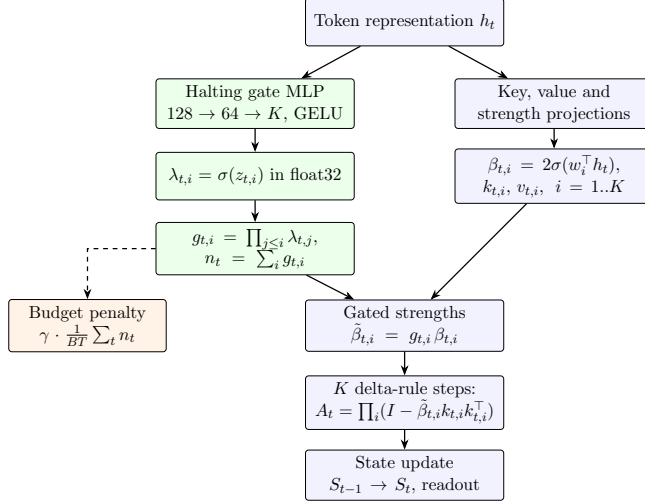


Figure 3.5.1: The adaptive-order layer. The halting gate (green) emits monotone gates that scale the strength of each Householder factor; a factor whose gate is zero is the identity. The effective order n_t enters the loss through the budget penalty (orange) during training.

the model first learns the task, and then rises linearly over $s_r = 4000$ steps to its final weight γ (Algorithm 2 in Appendix I).

Initialisation and precision. The gate’s output layer starts with zero weights and bias 4, so the initial effective order is $\sum_{i=1}^4 \sigma(4)^i = 3.8233$ for every token: the gate starts open and must learn to close. The recurrence kernels run in bfloat16, in which a sigmoid rounds to exactly 1.0, with zero gradient, for every logit above 6.2364; the gate is therefore evaluated in float32, which moves this threshold to 17.33 (Table A.5.1, Appendix I).

3.6 Reference Allocation

To measure what exact per-token allocation achieves independently of whether it can be learned, the gate is replaced by a *reference allocation* (the oracle), $g_{t,i}^{\text{oracle}} = \mathbf{1}[i \leq \ell(x_t)]$. For alphabets generating S_5 this equals $D(x_t)$. The oracle uses the same layer and training procedure as the learned model; only the source of the gates differs. It reads the target structure and serves only as a reference.

CHAPTER 4

EXPERIMENTAL RESULTS AND DISCUSSION

4.1 Experimental Setup and Metrics

Table 4.1.1 lists the configuration shared by all experiments; deviations are stated where they occur. Every run uses a single layer, so that the order of that layer is the only source of expressivity. All experiments ran on one NVIDIA RTX 4050 Laptop GPU with 6 GB of memory; the software environment is recorded in Table A.5.2 (Appendix I), with package versions pinned so that results from different dates remain comparable.

Table 4.1.1: Training configuration

Setting	Value
Layers; hidden size	1; 128
Heads $\times$ head dimension	12×32
Maximum order K (fixed-order arms)	4 (1, 2, 3, 4)
Output classes	127 (120 group elements, 7 special tokens)
Sequence length	128
Training / test sequences	100 000 / 2 000
Batch size; training steps	128; 20 000
Optimiser; learning rate	AdamW; 10^{-3} , constant
Recurrence precision	bfloat16
Gate hidden width; precision	64; float32
Gate initial bias b_0	4.0
Budget warm-up s_w ; ramp s_r	4 000; 4 000 steps
Penalty weight γ	0, 0.01, 0.03, 0.1, 0.3
Extended runs	250 000 steps, 300 000 sequences, 5 000-step warm-up, cosine decay

Token accuracy is the fraction of positions predicted correctly; sequence accuracy is the fraction of sequences correct at every position. To describe length generalisation, the per-position sequence accuracy $\text{Acc}_{\text{seq}}(\tau)$ records the fraction of sequences

still entirely correct up to position τ . The *required order* $\bar{n}$ is the mean number of factors applied per token with hard gates, and is compared with $\mathbb{E}[D]$, the order an exact allocation requires. A run *recovers the demand structure* if its allocation correlates with D at 0.99 or above while its token accuracy is at least 0.99; *under-allocation* is the fraction of tokens given fewer factors than they require.

Training on these tasks is bistable: a run either converges close to perfect accuracy or remains on a plateau near chance. The bfloat16 kernels accumulate in a non-deterministic order, so runs with the same configuration and random seed can reach different outcomes. Where outcomes are bistable, results are therefore reported as the *escape fraction* k/n , the number of runs out of n whose token accuracy reaches 0.90, with a 95% Wilson score interval. For configurations whose outcomes spread continuously, all outcomes are reported instead.

4.2 Reproduction of the Base Method

DeltaProduct was first reproduced with the authors’ training code, before any modification (Table 4.2.1). On S_3 the reproduction matches the published pattern: two factors reach sequence accuracy 1.000 within one epoch, while one factor stays at zero, as the rank bound predicts for the 3-cycles of S_3 . On S_5 , a single four-factor layer (12 heads of dimension 32, 500 000 sequences, 100 epochs, 7 h 45 min) was trained at length 128 and evaluated at length 512.

Table 4.2.1: Reproduction of DeltaProduct on the S_3 and S_5 word problems (the S_5 result at length 128 is measured on the first 128 positions of the length-512 evaluation)

Group	n_h	Train length	Eval. length	Token acc.	Sequence acc.
S_3	1	128	128	—	0.000
S_3	2	128	128	—	1.000
S_5	4	128	128	—	0.998
S_5	4	128	512	0.8644	0.000

Figure 4.2.1 shows the per-position sequence accuracy. The model is exact on every sequence through position 100 and on 99.8% at the training length of 128. Beyond it, accuracy decays gradually rather than collapsing: 50.5% of sequences are still entirely correct at position 250, 16.0% at 300, and none by position 443. Token

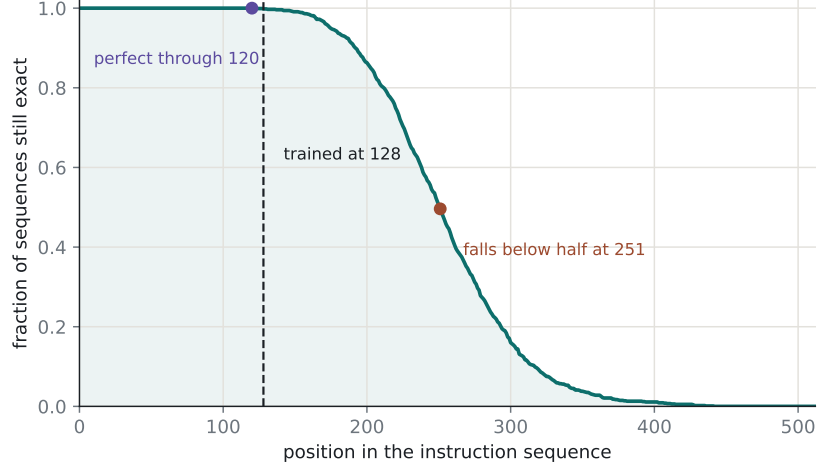


Figure 4.2.1: Length generalisation of the reproduced DeltaProduct model on S_5 ($n_h = 4$, one layer). The curve gives the fraction of test sequences still entirely correct up to each position (Section 4.1); the dashed line marks the training length of 128.

accuracy over all 512 positions is 0.8644. The full-length sequence accuracy is zero only because every sequence eventually accumulates an error, which is why the curve is a more informative summary than a single number. This graceful degradation matches the behaviour reported for DeltaProduct. Preliminary runs with a smaller dataset and budget remained at chance for every order, confirming that the published training budget is necessary.

4.3 Verification of the Order-Requirement Analysis

Every claim of Section 3.4 was verified by explicit construction in double precision, without any optimiser or training (Table 4.3.1). The rank and parity bounds hold for all 120 elements of S_5 ; the resulting reachability pattern is given in Table A.1.2 (Appendix I). No faithful representation of S_5 , of any dimension, lowers the requirement of a 5-cycle below four, and the best three-factor approximation of a 5-cycle is 68% away in relative norm. The constructed icosahedral rotations are exactly A_5 , and every non-identity element, including the 5-cycles, is a product of two reflections.

Within A_5 a 5-cycle therefore requires two factors; within S_5 it requires four.

Table 4.3.1: Verification of the analysis by explicit construction (float64)

Check	Result
$\text{rank}(A - I) \leq K$ for products of K factors, all β	Holds in every case tested
$\text{rank}(P_g - I) = \ell(g)$ for every $g \in S_5$	Holds for all 120 elements
Free β : P_g reachable with K factors iff $K \geq \ell(g)$	Holds; worst error 1.17×10^{-15}
Pinned $\beta = 2$: reachable iff $K \geq \ell(g)$ and $K \equiv \ell(g) \pmod{2}$	Holds for all 120 elements
Faithful representations of S_5 : $\text{rank}(\rho(c) - I)$ for a 5-cycle c	Always a multiple of 4; minimum 4; regular rep. 96
Best 3-factor approximation of a 5-cycle in S_5	68% relative error
Icosahedral rotations: count; orthogonality error	60 distinct; 1.39×10^{-16}
Element-order profile of the rotations	$\{1:1, 2:15, 3:20, 5:24\}$, that of A_5
Isomorphism onto A_5 via the five inscribed octahedra	Faithful, all images even, onto A_5 , homomorphic
Every non-identity rotation as two reflections	Holds for all 59; worst error 1.31×10^{-15}

4.4 Capability at Matched Compute

This experiment tests H1 on the mixture alphabets, where every token is a transposition ($D = 1$) or a 5-cycle ($D = 4$). For $p < 1$ the alphabet generates S_5 , so a fixed-order model needs four factors, while an exact allocation pays $\mathbb{E}[D] = 1 + 3p$. Four fixed-order arms and the reference allocation were trained under identical conditions, one run per cell (Figure 4.4.1, Table 4.4.1; sequence accuracies in Table A.2.1, Appendix I).

For $p = 0.05, 0.10$ and 0.25 the reference allocation matches the four-factor model while requiring 1.150, 1.300 and 1.752 factors per token, 3.5, 3.1 and 2.3 times fewer, and no fixed order below four solves these tasks. These results support H1. At $p = 0.5$ neither arm converged within 20 000 steps (losses were still falling), so that cell is limited by the training budget. At $p = 1$ the demand does not vary and the reference

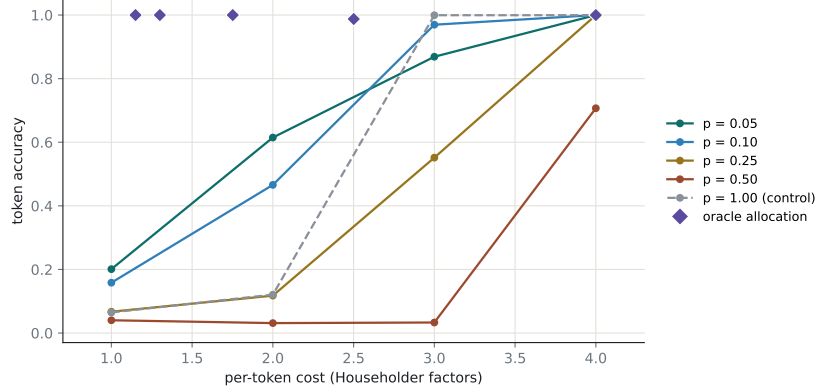


Figure 4.4.1: Token accuracy against required order per token. Each line connects the fixed-order arms $n_h = 1$ to 4 at one mixture rate; diamonds mark the reference allocation, which reaches the accuracy of $n_h = 4$ at a far lower required order for $p \leq 0.25$. At $p = 1$ it coincides with $n_h = 4$.

Table 4.4.1: Token accuracy at matched compute (one run per cell)

p	$\mathbb{E}[D]$	$n_h = 1$	$n_h = 2$	$n_h = 3$	$n_h = 4$	Reference
0.05	1.150	0.2009	0.6148	0.8689	0.9999	1.0000
0.10	1.300	0.1583	0.4659	0.9696	0.9999	0.9999
0.25	1.752	0.0671	0.1176	0.5516	0.9997	0.9999
0.50	2.501	0.0402	0.0313	0.0332	0.7070	0.9875
1.00	4.000	0.0648	0.1207	0.9994	0.9995	0.9998

coincides with four factors, as intended; the three-factor model also solves this task (0.9994) because an alphabet of 5-cycles generates A_5 , where a 5-cycle requires only two factors. This observation motivated the analysis of Section 3.4. Intermediate values such as 0.5516 are single runs of possibly bistable configurations (Section 4.5) and are individual outcomes, not capability measurements.

Figure 4.4.2 relates required order to measured training time at $p = 0.10$. For fixed orders, time grows with the number of factors, from 347 s at one factor to 761 s at four, confirming that the factor count drives the layer’s cost when every factor is executed. The reference allocation requires 1.300 factors per token yet takes 756 s, within 1% of four factors, because the current implementation still executes every factor and gives unused ones zero strength. The reductions reported in this chapter are therefore reductions in required factors per token; realising them in time requires

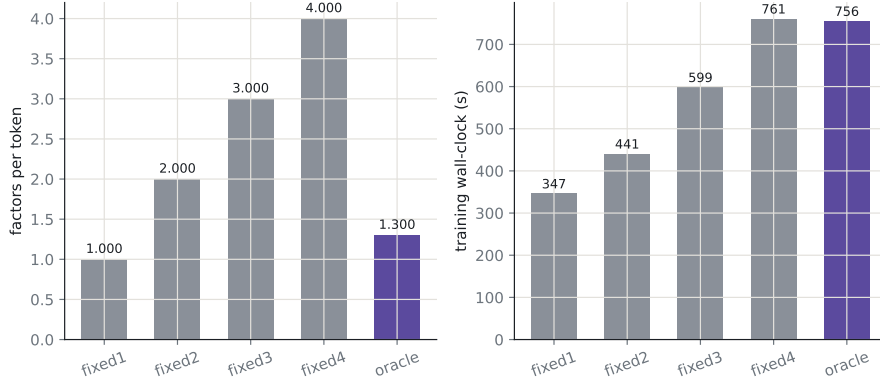


Figure 4.4.2: Required order per token (left) and measured training time (right) for the arms at $p = 0.10$. Training time grows with the order of the fixed-order arms, but the reference allocation takes as long as the four-factor model because unused factors are still executed with zero strength.

executing only the allocated factors (Section 5.2).

4.5 Alphabet Closure and Training Dynamics

This experiment tests H2 by holding the task format fixed and changing only the alphabet. A single run per cell (Figure A.3.1, Table A.3.1, Appendix I) gives a sharp contrast at three factors: the A_5 alphabets reach 0.9991, 0.9994 and 0.9952 and the S_5 alphabets 0.1094 and 0.0307, and A5_c5_3c reaches 0.9510 with only two factors, which is impossible if the requirement were the transposition length.

Single runs are not reliable here, however. With identical configuration, seed and data, two runs of A5_c5 at two factors reached 0.9598 and 0.1177: the non-deterministic accumulation of the bfloat16 kernels decided the outcome, already visible after 5000 steps. Each cell was therefore repeated ten times (Figure 4.5.1, Table 4.5.1).

Four conclusions follow. **First, the A_5 alphabets train at two factors:** A5_c5 escapes in 3 of 10 runs and A5_c5_dt in 2 of 10, reaching 0.9728 and 0.9985, although the transposition length of their 5-cycles is four. Escape frequency is a matter of optimisation, but the existence of these runs, together with the construction of Section 4.3, confirms that two factors suffice when the alphabet generates A_5 , supporting H2. **Second, the single-run contrast between A_5 and S_5 does not hold:** S5_c5_t at

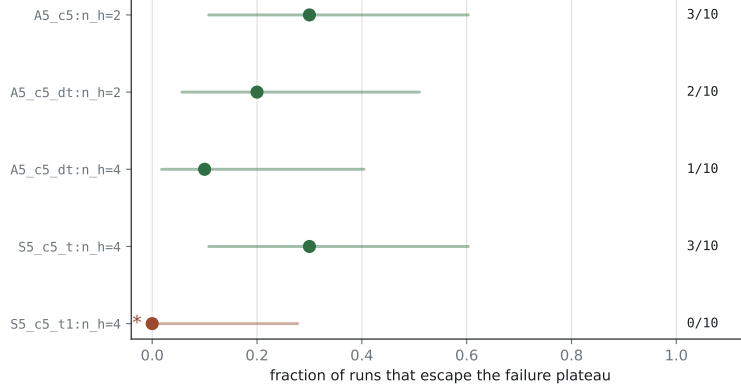


Figure 4.5.1: Escape fraction over ten runs for each alphabet and order, with 95% Wilson intervals. The marked configuration (S5_c5_t1) is not bimodal, so its fraction is shown only for completeness and is analysed separately in Figure A.3.2.

Table 4.5.1: Outcomes over ten runs per configuration (escape: token accuracy ≥ 0.90)

Configuration	Escapes	95% interval	Median	Best token	Best seq.
A5_c5, $n_h = 2$	3/10	[0.11, 0.60]	0.1199	0.9728	0.2945
A5_c5_dt, $n_h = 2$	2/10	[0.06, 0.51]	0.0435	0.9985	0.8695
A5_c5_dt, $n_h = 4$	1/10	[0.02, 0.40]	0.0400	0.9891	0.6090
S5_c5_t, $n_h = 4$	3/10	[0.11, 0.60]	0.0622	0.9994	0.9415
S5_c5_t1, $n_h = 4$	not bimodal		0.4845	0.7631	0.0790

four factors, 0.0706 in its single run, escapes in 3 of 10 runs, reaching 0.9994 with both even and odd targets resolved. An S_5 alphabet with 71% maximum-requirement tokens is therefore learnable at its required order. **Third, extra capacity does not make training reliable:** A5_c5_dt escapes in 2 of 10 runs at two factors and 1 of 10 at four. **Fourth, S5_c5_t1 behaves differently:** its ten runs spread continuously from 0.2453 to 0.7631 without forming two basins (Figure A.3.2, Appendix I), and a run with 12.5 times the steps and three times the data reached only 0.5661 (Table A.3.2). This configuration, with 96% maximum-requirement tokens, remains unresolved; it lies in the regime where per-token allocation has least to offer.

4.6 End-to-End Training of the Halting Gate

The halting gate was trained end to end on the mixture alphabet at $p = 0.10$, with no supervision on the allocation. The test tokens have two requirement classes, 90% transpositions ($D = 1$) and 10% 5-cycles ($D = 4$), so $\mathbb{E}[D] = 1.2985$. Five runs were made at each of $\gamma = 0.1$ and $\gamma = 0.3$ (Table 4.6.1, Figure 4.6.1; per-class allocation in Figure A.4.1, Appendix I).

Table 4.6.1: Learned allocation at $p = 0.10$, all runs ($\bar{n}$: mean factors per token with hard gates)

γ	Run	Token	Seq.	$\bar{n}$	$\bar{n} \mid D=1$	$\bar{n} \mid D=4$	Corr.	Under
0.1	1	0.99840	0.9405	3.9103	3.900	4.000	0.104	0.000
0.1	2	0.99985	0.9870	2.1990	2.000	4.000	1.000	0.000
0.1	3	0.99896	0.9260	4.0000	4.000	4.000	0.000	0.000
0.1	4	0.99962	0.9610	4.0000	4.000	4.000	0.000	0.000
0.1	5	0.99993	0.9935	4.0000	4.000	4.000	0.000	0.000
0.3	1	1.00000	0.9995	3.8206	3.801	4.000	0.155	0.000
0.3	2	0.99980	0.9805	2.1990	2.000	4.000	1.000	0.000
0.3	3	0.99956	0.9640	3.0995	3.000	4.000	1.000	0.000
0.3	4	0.99978	0.9830	4.0000	4.000	4.000	0.000	0.000
0.3	5	0.99999	0.9995	4.0000	4.000	4.000	0.000	0.000

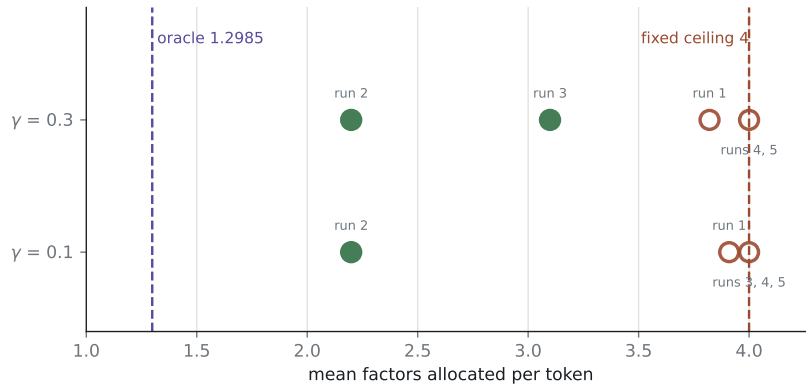


Figure 4.6.1: Mean factors allocated per token by each run of the learned gate. Filled markers are runs whose allocation separates the two requirement classes; the dashed lines mark the exact allocation (1.2985) and the fixed maximum (4).

The gate never harms the task: token accuracy is at least 0.998 in all ten

runs, no token is ever under-allocated, and 5-cycles receive all four factors in every run. **Three runs recover the demand structure:** two assign exactly two factors to every transposition and four to every 5-cycle (a mean of 2.1990 factors per token at token accuracies of 0.99985 and 0.99980), and a third assigns three and four (3.0995). **The remaining runs do not close:** five keep all four factors for every token and two close only partially (3.91 and 3.82). The escape fraction is thus $2/5$ at $\gamma = 0.1$ and $3/5$ at $\gamma = 0.3$ by the criterion of a mean order below four with accuracy above 0.99, and $3/10$ overall by the stricter criterion of recovering the structure. The best runs are both cheaper and more accurate than the three-factor model (3 factors, 0.9696) and approach the four-factor model (4 factors, 0.9999); the remaining gap to the exact allocation lies entirely in the low-requirement tokens, which receive two factors instead of one. Because only two requirement values occur, the correlation shows that the classes are separated, not that a multi-valued requirement would be matched level by level.

Table 4.6.2: All learned-gate runs by training configuration

Configuration	γ	Runs	Recovered	Lowest $\bar{n}$ (token)
Initial configuration	0, 0.01, 0.03, 0.1	12	2	1.2985 (0.99990)
Gate re-initialised	0.03	10	0	—
Initial-bias sweep	0.03	7	1	2.1990 (0.99961)
Float32 halting	0.03	10	1	2.1990 (0.99984)
Penalty-weight sweep	0.1, 0.3	10	3	2.1990 (0.99985)
Total		49	7	

Table 4.6.2 places the sweep among all configurations in which the gate was trained (see also Figure A.4.2, Appendix I). Across 49 learned runs, 7 recovered the demand structure. In the initial configuration, two runs, at $\gamma = 0.03$ and $\gamma = 0.1$, allocated exactly one factor to every transposition and four to every 5-cycle, reaching the exact allocation of 1.2985 factors per token with token accuracies of 0.99990 and 0.99998. The gate can therefore recover the requirement exactly, including the single factor needed by low-requirement tokens. Later configurations have not reproduced this, and identifying which change raised the floor to two factors is the first priority for future work.

4.7 Discussion

Table 4.7.1: Summary of the evidence for each hypothesis

Hypothesis	Evidence	Status
H1: per-token allocation reaches maximal-order accuracy at lower order	Reference allocation matches four factors at 1.150, 1.300 and 1.752 factors per token for $p \leq 0.25$ (single runs)	Supported
H2: requirement set by the alphabet’s closure	Proved and verified by construction; A_5 alphabets train at two factors in 3/10 and 2/10 runs, below the transposition length of their five-cycles	Supported
H3: a learned gate recovers the requirement	3 of 10 runs in the penalty sweep and 7 of 49 overall separate the classes with no under-allocation; one run reaches the exact allocation	Partially supported

Table 4.7.1 relates the results to the hypotheses. On the capability side, the central finding is that a token’s requirement is a property of the alphabet it belongs to: the same 5-cycle requires two factors when the closure is A_5 and four when it is S_5 . The requirement therefore cannot be read off a token in isolation, and neither can the correct fixed order be chosen by inspecting the inputs one at a time, which is precisely the situation in which a learned allocation is useful. The analysis also predicts where allocation cannot help, namely alphabets whose closure is A_4 or A_5 .

On the learning side, the dominant factor in every experiment is the bistability of training: both the base layer and the halting gate either reach a clean solution or remain on a plateau, with the outcome decided early and not fixed by the random seed. Conclusions have therefore been drawn from repeated runs or from the existence of successful runs, not from single outcomes. When the gate succeeds, it succeeds cleanly, separating the requirement classes exactly with no under-allocation and no loss of accuracy. The practical usefulness of the mechanism now depends on raising the fraction of runs that succeed and on executing only the allocated factors, so that the reduction in required order becomes a reduction in time.

CHAPTER 5

CONCLUSION AND FUTURE WORK

5.1 Conclusion

This project set out to replace the fixed, global number of Householder factors in DeltaProduct with a number chosen per token by a learned mechanism, and to establish how many factors each token requires. The base method was reproduced on a single 6 GB GPU: the S_3 results match the published pattern exactly, and the S_5 model is exact on 99.8% of sequences at its training length and reaches a token accuracy of 0.864 at four times that length.

The analysis established that a product of K factors can move at most K dimensions, that factors of fixed reflection strength can only be removed in pairs, and, centrally, that the requirement of a token is set by the cheapest faithful representation of the group generated by the whole alphabet rather than by the token. A 5-cycle requires four factors when the closure is S_5 and two when it is A_5 , and alphabets generating A_5 are indeed learned with two factors. With a reference allocation derived from this analysis, per-token order matches the four-factor model while requiring 2.3 to 3.5 times fewer factors per token. A monotone halting gate trained end to end without supervision on the allocation recovered the demand structure exactly in 7 of 49 runs overall, never under-allocated a token and never reduced accuracy; its best runs require 2.20 factors per token at a token accuracy of 0.9998, and one configuration reached the exact allocation of 1.30.

Two findings define the current state of the work. Training in this setting is bistable, so reliability, measured as the fraction of runs that succeed, is now the main limitation of the mechanism. And the reported savings are in required factors per token: the current implementation still executes every factor, so no reduction in time has yet been realised.

5.2 Future Work

- (1) **Reliability of the learned gate.** Compare the earliest gate configuration, which reached the exact allocation, with later ones one change at a time to find what raised the floor to two factors, and measure the success fraction over enough repeated runs to give narrow confidence intervals.
- (2) **Realised efficiency.** Execute only the allocated factors, for example with a per-level capacity router in the spirit of Mixture-of-Depths [14] that keeps tensor shapes static, and measure training and inference time against fixed orders at equal accuracy.
- (3) **Multi-level requirements.** Evaluate the gate on alphabets whose requirement takes every value from 1 to 4, and test whether it learns that a 5-cycle requires two factors under an A_5 alphabet but four under an S_5 one.
- (4) **The high-requirement regime.** Test the removal of the additive input pathway [17] on the configuration with 96% maximum-requirement tokens, which remains unresolved.
- (5) **Natural-language state tracking.** Once training is reliable, evaluate the layer on benchmarks such as SCONE [21] and entity tracking [22], where the number of state components an instruction changes gives an independent check on the learned allocation.

APPENDIX I

This appendix collects supplementary results referred to in the main chapters.

A.1 Theoretical Results

Table A.1.1 applies Definition 3.1 to the groups used in state-tracking benchmarks. Per-token allocation can help only where the requirement varies across non-identity elements, which excludes A_4 and A_5 . Table A.1.2 gives the exact reachability pattern behind Proposition 3.2, verified on all 120 elements of S_5 .

Table A.1.1: Order requirement across groups

Group	$ G $	Min. faithful dim.	D (non-identity)	ℓ range	D varies?
S_3	6	2	1, 2	1–2	yes
S_4	24	3	1, 2, 3	1–3	yes
A_4	12	3	2	2	no
A_5	60	3	2	2–4	no
S_5	120	4	1, 2, 3, 4	1–4	yes

Table A.1.2: Exact reachability of P_g with K factors, by transposition length ℓ ($\checkmark$ reachable)

ℓ	Free $\beta \in [0, 2]$					Pinned $\beta = 2$				
	$K=0$	1	2	3	4	$K=0$	1	2	3	4
0	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	–	$\checkmark$	–	$\checkmark$
1	–	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	–	$\checkmark$	–	$\checkmark$	–
2	–	–	$\checkmark$	$\checkmark$	$\checkmark$	–	–	$\checkmark$	–	$\checkmark$
3	–	–	–	$\checkmark$	$\checkmark$	–	–	–	$\checkmark$	–
4	–	–	–	–	$\checkmark$	–	–	–	–	$\checkmark$

A.2 Matched-Compute Sequence Accuracy

Table A.2.1 complements Table 4.4.1 with sequence accuracy, which counts a sequence as correct only if all 128 positions are correct.

Table A.2.1: Sequence accuracy at matched compute (one run per cell)

p	$n_h = 1$	$n_h = 2$	$n_h = 3$	$n_h = 4$	Reference
0.05	0.0005	0.0220	0.4020	0.9945	0.9960
0.10	0.0000	0.0005	0.6645	0.9915	0.9970
0.25	0.0000	0.0000	0.0020	0.9775	0.9960
0.50	0.0000	0.0000	0.0000	0.0025	0.4095
1.00	0.0000	0.0000	0.9535	0.9600	0.9825

A.3 Alphabet-Closure Details

Figure A.3.1 and Table A.3.1 show the single-run grid of alphabets and orders discussed in Section 4.5. Figure A.3.2 and Table A.3.2 document the configuration whose outcomes do not separate into two basins, and the effect of extended training on it.

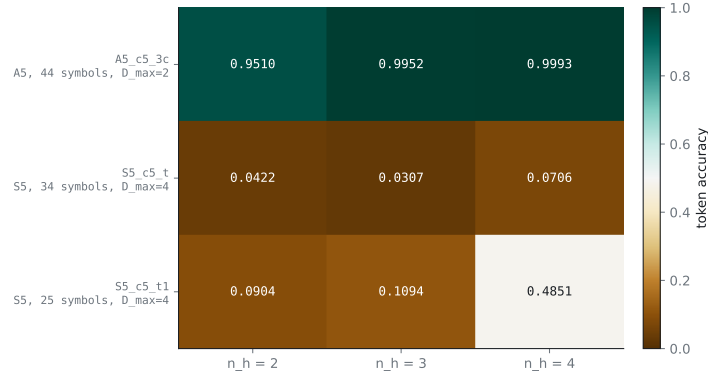


Figure A.3.1: Token accuracy for three closure alphabets at orders 2, 3 and 4, one run per cell. The A_5 alphabet trains at two factors, below the transposition length of its five-cycles.

Table A.3.1: Token accuracy by alphabet and order (one run per cell; sequence accuracy in parentheses)

Alphabet	$\langle \Sigma \rangle$	$D_{\max}$	$n_h = 2$	$n_h = 3$	$n_h = 4$
A5_c5	A_5	2	–	0.9991 (0.936)	–
A5_c5_dt	A_5	2	–	0.9994 (0.953)	–
A5_c5_3c	A_5	2	0.9510 (0.135)	0.9952 (0.751)	0.9993 (0.953)
S5_c5_t1	S_5	4	0.0904 (0.000)	0.1094 (0.000)	0.4851 (0.025)
S5_c5_t	S_5	4	0.0422 (0.000)	0.0307 (0.000)	0.0706 (0.000)

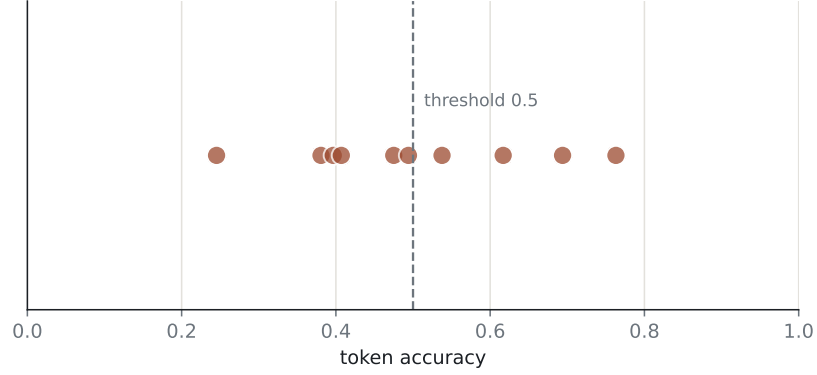


Figure A.3.2: Token accuracy of ten runs of S5_c5_t1 at four factors. The outcomes spread continuously without separating into a low and a high basin.

Table A.3.2: S5_c5_t1: token accuracy under extended training (one run per cell)

Training	Sequences	$n_h = 2$	$n_h = 3$	$n_h = 4$
20 000 steps, constant LR	100 000	0.0904	0.1094	0.4851
40 000 steps, cosine schedule	100 000	–	0.4749	0.4843
80 000 steps	100 000	–	–	0.4721
250 000 steps, warm-up and cosine	300 000	0.3617	0.4828	0.5661
Ten runs, 20 000 steps: median (range)	100 000	–	–	0.4845 (0.245–0.763)

A.4 Learned-Gate Details

Figure A.4.1 shows, for every run of the penalty sweep, the factors given to each requirement class. Figure A.4.2 summarises all learned-gate configurations of Table 4.6.2.

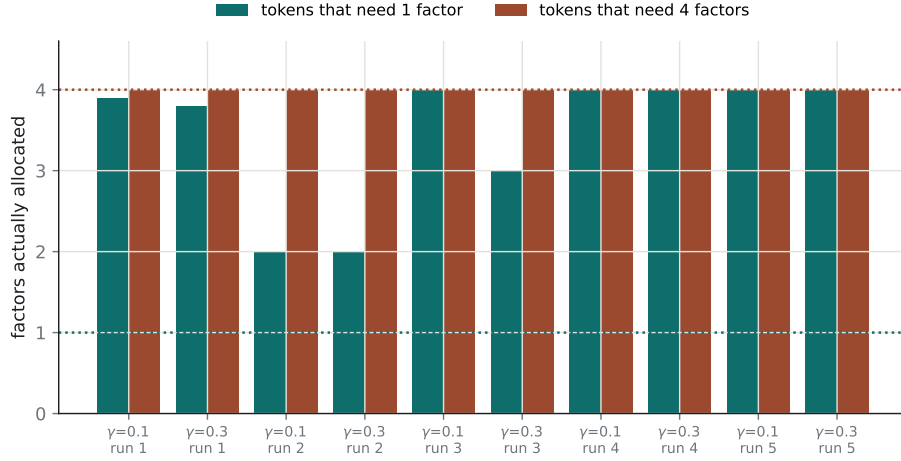


Figure A.4.1: Factors allocated to low-requirement tokens ($D = 1$) and high-requirement tokens ($D = 4$) in each run. Dotted lines mark the exact allocation for each class; high-requirement tokens receive all four factors in every run.

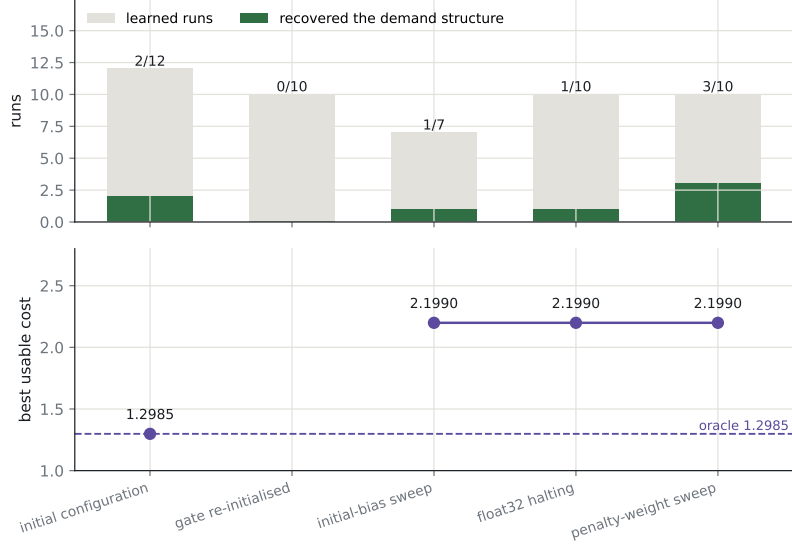


Figure A.4.2: Learned-gate runs across training configurations: the number of runs and the number that recovered the demand structure (top), and the lowest mean order among recovered runs (bottom), against the exact allocation of 1.2985.

A.5 Implementation Details

Table A.5.1 verifies the choice of float32 for the halting nonlinearity. A gate whose logits were set to 10, beyond the bfloat16 threshold, was optimised for 300 steps on the budget penalty alone. In bfloat16 it receives a gradient of exactly zero and never moves; in float32 it closes from 3.9995 to 0.0625 factors. With small logits both precisions behave identically, so the effect is specific to saturation. The gate initialisation was verified in the same code path used for training: with the model class’s default initialisation the initial effective order was 0.9376, whereas with explicit initialisation it was 3.8233 for every token, and every run in Table 4.6.1 logged 3.823 at its first step. Algorithm 1 gives the forward pass of the adaptive-order layer and Algorithm 2 the training procedure, and Table A.5.2 records the environment.

Table A.5.1: Effect of the precision of the halting nonlinearity

Quantity	bfloat16	float32
Largest representable value below 1.0	0.99609375	0.99999994
Logit above which $\sigma(z)$ rounds to 1.0	6.2364	17.3287
Gradient norm, saturated gate (logit 10)	0 exactly	3.818×10^{-3}
Effective order after 300 penalty-only steps	4.0000 $\rightarrow$ 4.0000	3.9995 $\rightarrow$ 0.0625
Gradient norm, responsive gate (logit 1)	9.359	9.304
Effective order after 300 penalty-only steps	1.9375 $\rightarrow$ 0.0000	1.9419 $\rightarrow$ 0.0000

Algorithm 1: Forward pass of the adaptive-order layer for one token

Input: token representation h_t ; previous state S_{t-1} ; maximum order K ;
mode (training or evaluation)

Output: new state S_t ; effective order n_t

```

1  $z_t \leftarrow W_2 \text{GELU}(W_1 h_t + b_1) + b_2$  ; //  $K$  gate logits
2  $\lambda_{t,i} \leftarrow \sigma(z_{t,i})$  for  $i = 1, \dots, K$  ; // evaluated in float32
3  $g_{t,i} \leftarrow \prod_{j=1}^i \lambda_{t,j}$  ; // monotone gates
4 if  $mode = \text{evaluation}$  then
5   |  $g_{t,i} \leftarrow \mathbf{1}[g_{t,i} > 0.5]$  ; // hard gates, integer order
6 end
7  $n_t \leftarrow \sum_{i=1}^K g_{t,i}$ 
8  $S \leftarrow S_{t-1}$ 
9 for  $i \leftarrow 1$  to  $K$  do
10 |  $\tilde{\beta}_{t,i} \leftarrow g_{t,i} \cdot 2 \sigma(w_i^\top h_t)$ ;
11 |  $S \leftarrow S (I - \tilde{\beta}_{t,i} k_{t,i} k_{t,i}^\top) + \tilde{\beta}_{t,i} v_{t,i} k_{t,i}^\top$ ;
12 end
13 return  $S_t \leftarrow S, n_t$ ;
```

Algorithm 2: Training the adaptive-order layer with a compute budget

Input: training sequences; penalty weight γ ; warm-up s_w ; ramp s_r ; number of steps N
Output: trained model

- 1 Initialise the gate output layer with zero weights and bias b_0 ;
- 2 **for** $s \leftarrow 1$ **to** N **do**
- 3 Sample a batch of B sequences of length T ;
- 4 Run Algorithm 1 in training mode over every position;
- 5 $\mathcal{L}_{\text{task}} \leftarrow$ cross-entropy of the readout against $y_{b,t}$;
- 6 $\mathcal{L} \leftarrow \mathcal{L}_{\text{task}} + \gamma(s) \cdot \frac{1}{BT} \sum_{b,t} n_{b,t}$; // Equation 3.5
- 7 Update all parameters with AdamW;
- 8 **end**
- 9 Evaluate with hard gates on the held-out sequences;

Table A.5.2: Hardware and software environment

Component	Version
GPU	NVIDIA GeForce RTX 4050 Laptop, 6 GB VRAM
Operating system	Ubuntu 26.04 on WSL2, Windows host
Python	3.12.7
PyTorch; CUDA; cuDNN	2.6.0; 12.6; 9.5.1
Triton	3.2.0
Transformers	4.49.0
flash-linear-attention	repository copy with the modifications of Section 3.5
causal-conv1d	1.7.0

KEY REFERENCES

- [1] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are RNNs: Fast autoregressive transformers with linear attention. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, pages 5156–5165, 2020.
- [2] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. In *First Conference on Language Modeling (COLM)*, 2024.
- [3] William Merrill, Jackson Petty, and Ashish Sabharwal. The illusion of state in state-space models. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024.
- [4] Yash Sarrof, Yana Veitsman, and Michael Hahn. The expressive capacity of state space models: A formal language perspective. In *Advances in Neural Information Processing Systems 37 (NeurIPS)*, 2024.
- [5] Mehran Shakerinava, Behnoush Khavari, Siamak Ravanbakhsh, and Sarath Chandar. The expressive limits of diagonal SSMS for state-tracking. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2603.01959.
- [6] Imanol Schlag, Kazuki Irie, and Jürgen Schmidhuber. Linear transformers are secretly fast weight programmers. In *Proceedings of the 38th International Conference on Machine Learning (ICML)*, pages 9355–9366, 2021.
- [7] Songlin Yang, Bailin Wang, Yu Zhang, Yikang Shen, and Yoon Kim. Parallelizing linear transformers with the delta rule over sequence length. In *Advances in Neural Information Processing Systems 37 (NeurIPS)*, 2024. arXiv:2406.06484.
- [8] Songlin Yang, Jan Kautz, and Ali Hatamizadeh. Gated delta networks: Improving Mamba2 with delta rule. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2412.06464.
- [9] Riccardo Grazi, Julien Siems, Arber Zela, Jörg K. H. Franke, Frank Hutter, and Massimiliano Pontil. Unlocking state-tracking in linear RNNs through negative eigenvalues. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2411.12537.
- [10] Julien Siems, Timur Carstensen, Arber Zela, Frank Hutter, Massimiliano Pontil, and Riccardo Grazi. DeltaProduct: Improving state-tracking in linear RNNs via Householder products. In *Advances in Neural Information Processing Systems 38 (NeurIPS)*, 2025. arXiv:2502.10297.
- [11] Alex Graves. Adaptive computation time for recurrent neural networks. arXiv preprint arXiv:1603.08983, 2016.
- [12] Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Łukasz Kaiser. Universal transformers. In *International Conference on Learning Representations (ICLR)*, 2019.

- [13] Andrea Banino, Jan Balaguer, and Charles Blundell. PonderNet: Learning to ponder. In *8th ICML Workshop on Automated Machine Learning (AutoML)*, 2021.
- [14] David Raposo, Sam Ritter, Blake Richards, Timothy Lillicrap, Peter Conway Humphreys, and Adam Santoro. Mixture-of-depths: Dynamically allocating compute in transformer-based language models. arXiv preprint arXiv:2404.02258, 2024.
- [15] Alston S. Householder. Unitary triangularization of a nonsymmetric matrix. *Journal of the ACM*, 5(4):339–342, 1958.
- [16] Samuel Larson. The rank the task demands: A causal rank law for matrix memories trained on group composition. arXiv preprint arXiv:2609.12259, 2026.
- [17] Gunner Levi Howe. The automaton underneath: The additive input pathway is a parasitic attractor for state tracking in Householder linear RNNs. arXiv preprint arXiv:2609.18966, 2026.
- [18] Tal Schuster, Adam Fisch, Jai Gupta, Mostafa Dehghani, Dara Bahri, Vinh Q. Tran, Yi Tay, and Donald Metzler. Confident adaptive language modeling. In *Advances in Neural Information Processing Systems 35 (NeurIPS)*, 2022.
- [19] Songlin Yang and Yu Zhang. FLA: A Triton-based library for hardware-efficient implementations of linear attention mechanism. <https://github.com/fla-org/flash-linear-attention>, 2024.
- [20] Emil Artin. *Geometric Algebra*. Interscience Publishers, New York, 1957.
- [21] Reginald Long, Panupong Pasupat, and Percy Liang. Simpler context-dependent logical forms via model projections. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2016.
- [22] Najoung Kim and Sebastian Schuster. Entity tracking in language models. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, 2023.

TIMELINE OF THE PROJECT

The project runs from September 2026 to April 2027. Table T.1 lists the completed and planned phases.

Table T.1: Project timeline

Period	Activity	Status
Sep 2026, weeks 1–2	Environment set-up; reproduction of DeltaProduct on S_3 and S_5	Completed
Sep 2026, weeks 2–3	Order-requirement analysis and verification; matched-compute and alphabet-closure experiments; repeated-run census	Completed
Sep 2026, weeks 3–4	Halting gate implementation; precision and initialisation verification; penalty-weight sweep; mid-term report	Completed
Oct 2026	Reliability of the learned gate: configuration comparison and repeated runs	Planned
Nov 2026	Multi-level requirement alphabets; dependence of the learned allocation on closure	Planned
Dec 2026 – Jan 2027	Execution of only the allocated factors; wall-clock measurement	Planned
Feb 2027	High-requirement regime; length generalisation; replication of headline results	Planned
Mar 2027	Evaluation on natural-language state-tracking benchmarks	Planned
Apr 2027	Final report and presentation	Planned